

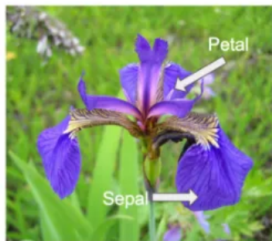
Decision Trees

Albert Y. Kim

Introduction

The famous statistician Ronald Fisher collected 50 samples from each of three species of Iris:

Iris setosa



Iris versicolor



Iris virginica

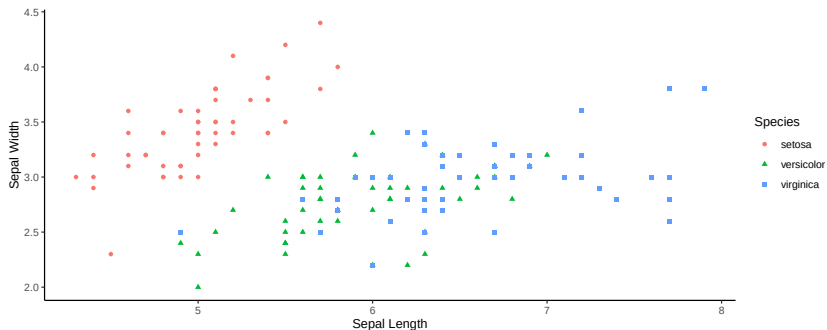


He measured the dimensions of the sepal and petal of each sample.

Image source: <https://peaceadegbite1.medium.com/iris-flower-classification-60790e9718a1>

Introduction

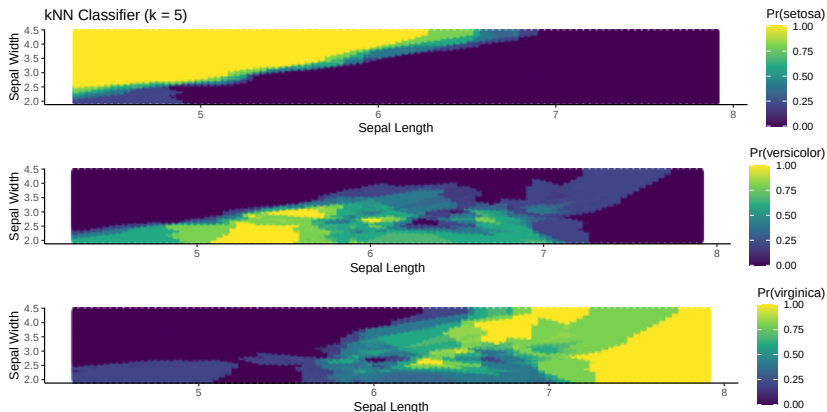
Shown below are the sepal dimensions of these flowers:



Can sepal dimensions be used to accurately classify different species of iris?

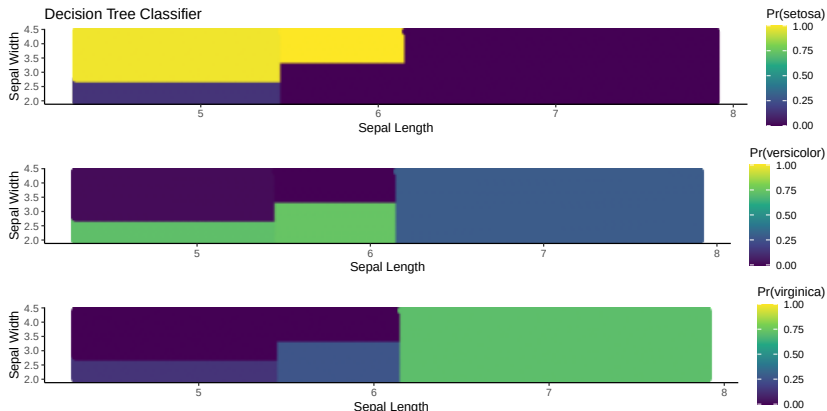
Classification with k -nearest neighbors

We can train a KNN classifier, but notice how the classification boundaries are highly irregular and seem somewhat haphazard:

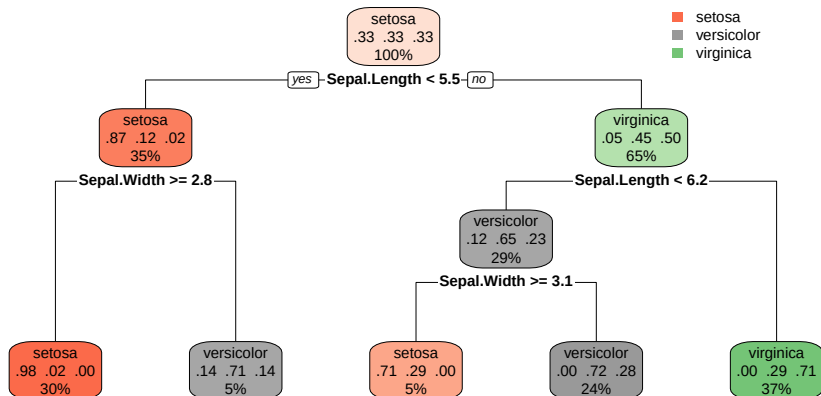


Decision trees on the iris data

Decision trees learn a series of recursive binary splitting rules to make classification decisions:



Example (full tree)



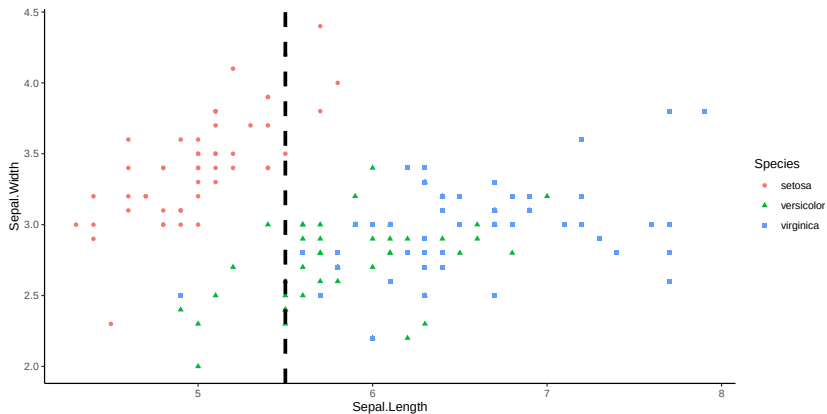
Decision trees

Decision trees are trained by recursively partitioning the p -dimensional space defined by the explanatory variables:

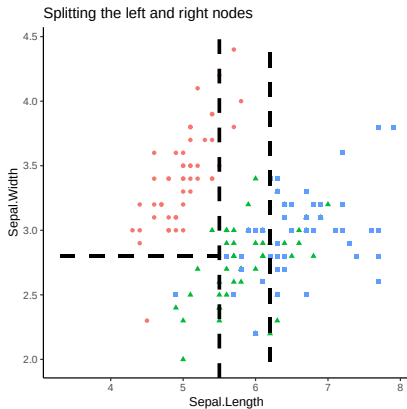
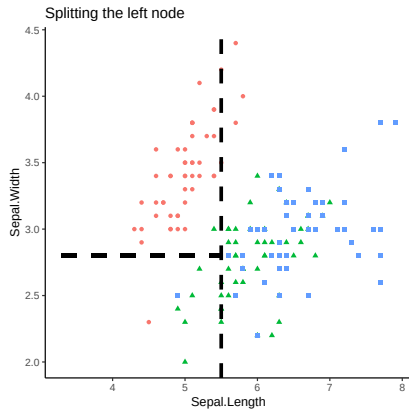
- 1) Starting with a “parent” node, search for a splitting rule that maximizes the *homogeneity* or *purity* of the “child” nodes
- 2) Next, considering each node that hasn't yet been split, find another splitting rule that maximizes *purity*
- 3) Repeat until a stopping criteria has been reached

We'll soon discuss how splitting rules are determined and how the algorithm terminates, but we'll first see what this looks like for the Iris data.

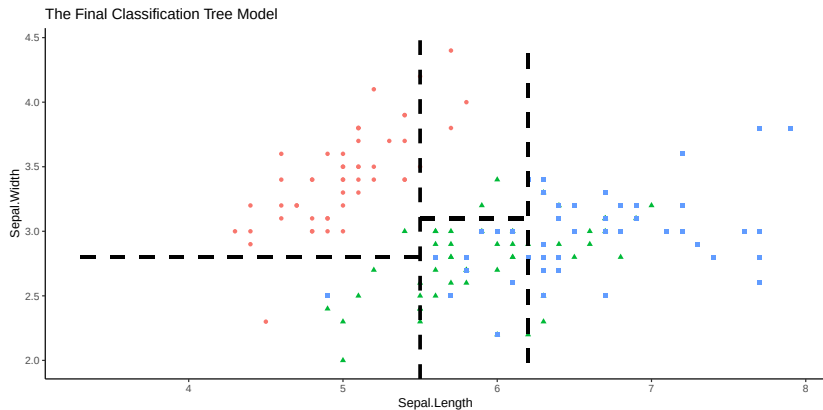
Example (first split)



Example (second split)



Example (final model)



Splitting criteria

Decision trees must learn their splits using an objective criteria, the most common criteria is *Gini impurity*:

$$Gini = \sum_{j=1}^k p_j(1 - p_j) = 1 - \sum_{j=1}^k p_j^2$$

- ▶ For binary classification, this reduces to $p_1(1 - p_1) + p_2(1 - p_2)$
- ▶ Other splitting criteria exist; however, Gini impurity is the default in `sklearn`

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

In our example tree:

- ▶ The Gini impurity of the:
 - ▶ starting node = $1 - (\frac{1}{3}^2 + \frac{1}{3}^2 + \frac{1}{3}^2) = \frac{2}{3} = 0.666$

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

In our example tree:

- ▶ The Gini impurity of the:
 - ▶ starting node = $1 - (\frac{1}{3}^2 + \frac{1}{3}^2 + \frac{1}{3}^2) = \frac{2}{3} = 0.666$
 - ▶ left child node = $1 - (0.87^2 + 0.12^2 + 0.02^2) = 0.23$
 - ▶ right child node = $1 - (0.05^2 + 0.45^2 + 0.50^2) = 0.62$

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

In our example tree:

- ▶ The Gini impurity of the:
 - ▶ starting node = $1 - (\frac{1}{3}^2 + \frac{1}{3}^2 + \frac{1}{3}^2) = \frac{2}{3} = 0.666$
 - ▶ left child node = $1 - (0.87^2 + 0.12^2 + 0.02^2) = 0.23$
 - ▶ right child node = $1 - (0.05^2 + 0.45^2 + 0.50^2) = 0.62$
- ▶ So the weighted average of the two child nodes was $0.35 * 0.23 + 0.65 * 0.62 = 0.434$, since
 - ▶ 35% of the original 150 irises are in the left child node
 - ▶ 65% of the original 150 irises are in the right child node

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

In our example tree:

- ▶ The Gini impurity of the:
 - ▶ starting node = $1 - (\frac{1}{3}^2 + \frac{1}{3}^2 + \frac{1}{3}^2) = \frac{2}{3} = 0.666$
 - ▶ left child node = $1 - (0.87^2 + 0.12^2 + 0.02^2) = 0.23$
 - ▶ right child node = $1 - (0.05^2 + 0.45^2 + 0.50^2) = 0.62$
- ▶ So the weighted average of the two child nodes was $0.35 * 0.23 + 0.65 * 0.62 = 0.434$, since
 - ▶ 35% of the original 150 irises are in the left child node
 - ▶ 65% of the original 150 irises are in the right child node
- ▶ So the Gini gain is $0.666 - 0.434 = 0.233$. i.e. the Gini impurity decreased by 0.233 closer towards 0 (perfect purity)

Decision Tree Learning

Building a tree involves iterating between two steps:

1. Finding the optimal splitting point within each variable
2. Selecting the variable to split on

Even if an exhaustive approach were taken, this can be done fairly quickly as there are $n - 1$ possible split points per variable and p variables

- ▶ Modern tree solving algorithms are beyond the scope of this course, but as you might image they do not check every possible split

Regression tasks (numerical outcomes)

Applying the decision tree algorithm to data with a numeric outcome involves the following changes:

- ▶ Predicted outcomes are the average outcome in a node
- ▶ Mean squared error is used instead of Gini impurity as a measure of impurity

Other splitting criteria, such as mean absolute error or Poisson deviance are available in `sklearn`

Stopping the algorithm

Decision trees can be grown until every terminal node is perfectly pure; however, such trees will be very overfit to the training data. We can manipulate the bias-variance trade-off in a fitted tree in the following ways:

1. Restricting the maximum depth of the tree (ie: the number of sequential rules)
2. Allowing only nodes of sufficient size be eligible for splitting
3. Requiring a certain improvement in purity for a split to occur

Generally, maximum depth is the most important factor to consider as it directly relates to the complexity of a tree

Practical guidance

We've now introduced two “classic” machine learning algorithms, KNN and decision trees:

- ▶ Decision trees
 - ▶ structured, but can still learn non-linear patterns
 - ▶ more easily interpretable
 - ▶ fairly robust to outliers and missing values
 - ▶ prone to overfitting at higher max depths
- ▶ KNN
 - ▶ very flexible, but produces irregular/inconsistent classification boundaries
 - ▶ sensitive to re-scaling
 - ▶ computationally inefficient with large data sets

Cross-validation provides an objective strategy for choosing between these models, but we should be aware of their strengths/weaknesses.

A Note About Today's Data

Run `?iris` in R, scroll down to Source, and see the journal this data was published in.